

UNIT Third :III: Basics of SQL

SQL statements and working with Tables

What is SQL:-

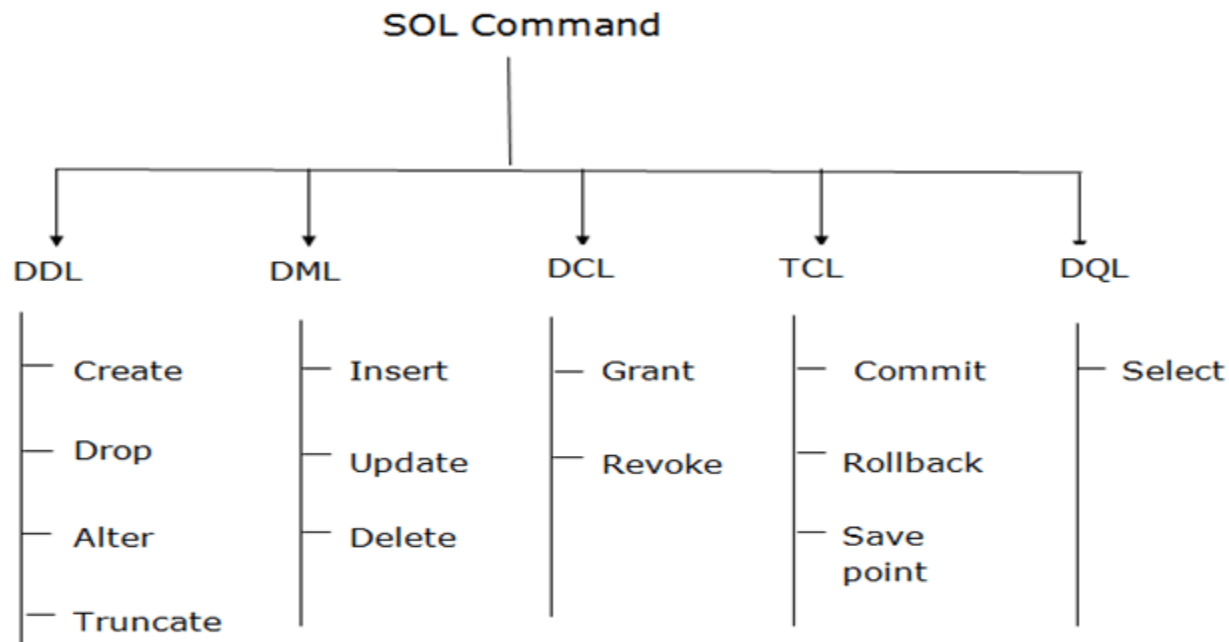
SQL is a short-form of the structured query language, and it is pronounced as S-Q-L or sometimes as See-Quell.

This database language is mainly designed for maintaining the data in relational database management systems. It is a special tool used by data professionals for handling structured data (data which is stored in the form of tables). It is also designed for stream processing in RDBMS.

You can easily create and manipulate the database, access and modify the table rows and columns, etc. This query language became the standard of ANSI in the year of 1986 and ISO in the year of 1987.

If you want to get a job in the field of data science, then it is the most important query language to learn. Big enterprises like Facebook, Instagram, and LinkedIn, use SQL for storing the data in the back-end.

SQL Commands



1. Data Definition Language (DDL)

- DDL changes the structure of the table like creating a table, deleting a table, altering a table, etc.
- All the command of DDL are auto-committed that means it permanently save all the changes in the database.
- Here are some commands that come under DDL:
- CREATE , DROP
- TRUNCATE , ALTER

a. CREATE It is used to create a new table in the database.

1. **Syntax:** CREATE TABLE TABLE_NAME (COLUMN_NAME DATATYPES[,....]);

Exa.:CREATE TABLE EMPLOYEE(Name VARCHAR2(20), Email VARCHAR2(100), DOB DATE);

b. DROP: It is used to delete both the structure and record stored in the table.

Syntax DROP TABLE table_name;

Example DROP TABLE EMPLOYEE;

c. ALTER: It is used to alter the structure of the database. This change could be either to modify the characteristics of an existing attribute or probably to add a new attribute.

Syntax: To add a new column in the table

ALTER TABLE table_name ADD column_name COLUMN-definition;

EXAMPLE ALTER TABLE STU_DETAILS ADD(ADDRESS VARCHAR2(20));

ALTER TABLE STU_DETAILS MODIFY (NAME VARCHAR2(20));

d. TRUNCATE: It is used to delete all the rows from the table and free the space containing the table.

Syntax: TRUNCATE TABLE table_name;

Example: TRUNCATE TABLE EMPLOYEE;

2. Data Manipulation Language(DML)

- DML commands are used to modify the database. It is responsible for all form of changes in the database.
- The command of DML is not auto-committed that means it can't permanently save all the changes in the database. They can be rollback.
- INSERT
- UPDATE
- DELETE

a. INSERT: The INSERT statement is a SQL query. It is used to insert data into the row of a table.

Syntax:

```
INSERT INTO TABLE_NAME  
(col1, col2, col3,.... col N)  
VALUES (value1, value2, value3, .... valueN);
```

Or

```
INSERT INTO TABLE_NAME  
VALUES (value1, value2, value3, .... valueN);
```

example:

```
INSERT INTO javatpoint (Author, Subject) VALUES ("Sonoo", "DBMS");
```

b. UPDATE: This command is used to update or modify the value of a column in the table.

Syntax:

```
UPDATE table_name SET [column_name1= value1,...column_nameN = valueN] [WHERE C  
ONDITION]
```

For example:

```
UPDATE students  
SET User_Name = 'Sonoo'  
WHERE Student_Id = '3'
```

c. DELETE: It is used to remove one or more row from a table.

Syntax: DELETE FROM table_name [WHERE condition];

For example:

```
DELETE FROM javatpoint
```

1. WHERE Author="Sonoo";

2. Data Control Language(DCL)

DCL commands are used to grant and take back authority from any database user.

Here are some commands that come under DCL:

- Grant , Revoke

a. Grant: It is used to give user access privileges to a database.

Example

```
GRANT SELECT, UPDATE ON MY_TABLE TO SOME_USER, ANOTHER_US  
ER;
```

b. Revoke: It is used to take back permissions from the user.

Example REVOKE SELECT, UPDATE ON MY_TABLE FROM USER1, USER2;

3. Transaction Control Language(TCL)

TCL commands can only use with DML commands like INSERT, DELETE and UPDATE only.

These operations are automatically committed in the database that's why they cannot be used while creating tables or dropping them.

Here are some commands that come under TCL:

- COMMIT , ROLLBACK ,SAVEPOINT

a. Commit: Commit command is used to save all the transactions to the database.

Syntax: COMMIT;

Example:

1. DELETE FROM CUSTOMERS
2. WHERE AGE = 25;
3. COMMIT;

b. Rollback: Rollback command is used to undo transactions that have not already been saved to the database.

Syntax: ROLLBACK;

Example:

1. DELETE FROM CUSTOMERS
2. WHERE AGE = 25;
3. ROLLBACK;

c. SAVEPOINT: It is used to roll the transaction back to a certain point without rolling back the entire transaction.

Syntax: SAVEPOINT SAVEPOINT_NAME;

SQL Data Types

Data types are used to represent the nature of the data that can be stored in the database table. For example, in a particular column of a table, if we want to store a string type of data then we will have to declare a string data type of this column.

Data types mainly classified into three categories for every database.

- String Data types, Numeric Data types , Date and time Data types
-
-

SQL Server String Data Type

char(n)	It is a fixed width character string data type. Its size can be up to 8000 characters.
varchar(n)	It is a variable width character string data type. Its size can be up to 8000 characters.
varchar(max)	It is a variable width character string data types. Its size can be up to 1,073,741,824 characters.
text	It is a variable width character string data type. Its size can be up to 2GB of text data.
nchar	It is a fixed width Unicode string data type. Its size can be up to 4000 characters.
nvarchar	It is a variable width Unicode string data type. Its size can be up to 4000 characters.
ntext	It is a variable width Unicode string data type. Its size can be up to 2GB of text data.
binary(n)	It is a fixed width Binary string data type. Its size can be up to 8000 bytes.
varbinary	It is a variable width Binary string data type. Its size can be up to 8000 bytes.

image	It is also a variable width Binary string data type. Its size can be up to 2GB.
--------------	---

- **SQL Server Numeric Data Types**

bit	It is an integer that can be 0, 1 or null.
tinyint	It allows whole numbers from 0 to 255.
Smallint	It allows whole numbers between -32,768 and 32,767.
Int	It allows whole numbers between -2,147,483,648 and 2,147,483,647.
bigint	It allows whole numbers between -9,223,372,036,854,775,808 and 9,223,372,036,854,775,807.
float(n)	It is used to specify floating precision number data from -1.79E+308 to 1.79E+308. The n parameter indicates whether the field should hold 4 or 8 bytes. Default value of n is 53.
real	It is a floating precision number data from -3.40E+38 to 3.40E+38.
money	It is used to specify monetary data from -922,337,233,685,477.5808 to 922,337,203,685,477.5807.

- **SQL Server Date and Time Data Type**

datetime	It is used to specify date and time combination. It supports range from January 1, 1753, to December 31, 9999 with an accuracy of 3.33 milliseconds.
-----------------	--

datetime2	It is used to specify date and time combination. It supports range from January 1, 0001 to December 31, 9999 with an accuracy of 100 nanoseconds
date	It is used to store date only. It supports range from January 1, 0001 to December 31, 9999
time	It stores time only to an accuracy of 100 nanoseconds
timestamp	It stores a unique number when a new row gets created or modified. The time stamp value is based upon an internal clock and does not correspond to real time. Each table may contain only one time stamp variable.

- **SQL Server Other Data Types**

Sql_variant	It is used for various data types except for text, timestamp, and ntext. It stores up to 8000 bytes of data.
XML	It stores XML formatted data. Maximum 2GB.
cursor	It stores a reference to a cursor used for database operations
table	It stores result set for later processing.
uniqueidentifier	It stores GUID (Globally unique identifier).

SQL CREATE TABLE

SQL CREATE TABLE statement is used to create table in a database.

If you want to create a table, you should name the table and define its column and each column's data type.

Let's see the simple syntax to create the table.

1. **create table** "tablename"
2. ("column1" "data type",
3. "column2" "data type",
4. "column3" "data type",
5. ...
6. "columnN" "data type");

The data type of the columns may vary from one database to another. For example, NUMBER is supported in Oracle database for integer value whereas INT is supported in MySQL.

Let us take an example to create a STUDENTS table with ID as primary key and NOT NULL are the constraint showing that these fields cannot be NULL while creating records in the table.

1. SQL> **CREATE TABLE** STUDENTS (
2. ID **INT** NOT NULL,
3. **NAME VARCHAR** (20) NOT NULL,
4. AGE **INT** NOT NULL,
5. ADDRESS **CHAR** (25),
6. **PRIMARY KEY** (ID)
7.);

You can verify it, if you have created the table successfully by looking at the message displayed by the SQL Server, else you can use DESC command as follows:

SQL> DESC STUDENTS;

FIELD	TYPE	NULL	KEY
ID	Int(11)	NO	PRI
NAME	Varchar(20)	NO	

AGE	Int(11)	NO		
ADDRESS	Varchar(25)	YES		

4 rows in set (0.00 sec)

SQL SELECT Statement

- The **SELECT** statement is the most commonly used command in Structured Query Language. It is used to access the records from one or more database tables and views. It also retrieves the selected data that follow the conditions we want.

Syntax of SELECT Statement in SQL

1. **SELECT** Column_Name_1, Column_Name_2,, Column_Name_N **FROM** Table_Name;
 In this SELECT syntax, **Column_Name_1, Column_Name_2,, Column_Name_N** are the name of those columns in the table whose data we want to read.
1. **SELECT * FROM** table_name;

Examples of SELECT Statement in SQL

Here, we took the following two different SQL examples which will help you to execute the SELECT statement for retrieving the records:

Example 1:

Firstly, we have to create the new table and then insert some dummy records into it.

```
CREATE TABLE Student_Records
(
  Student_Id Int PRIMARY KEY,
  First_Name VARCHAR (20),
  Address VARCHAR (20),
  Age Int NOT NULL,
  Percentage Int NOT NULL,
  Grade VARCHAR (10)
);
```

The following query inserts the record of intelligent students into the **Student_Records** table:

```
INSERT INTO Student VALUES (201, Akash, Delhi, 18, 89, A2),  
(202, Bhavesh, Kanpur, 19, 93, A1),  
(203, Yash, Delhi, 20, 89, A2),  
(204, Bhavna, Delhi, 19, 78, B1),  
(05, Yatin, Lucknow, 20, 75, B1),  
(206, Ishika, Ghaziabad, 19, 51, C1),  
(207, Vivek, Goa, 20, 62, B2);
```

The following SQL query displays all the values of each column from the above Student_records table:

1. **SELECT * FROM** Student_Records;

The output of the above query is:

Student_ID	First_Name	Address	Age
201	Akash	Delhi	18
202	Bhavesh	Kanpur	19
203	Yash	Delhi	20
204	Bhavna	Delhi	19
205	Yatin	Lucknow	20
206	Ishika	Ghaziabad	19
207	Vivek	Goa	20

SQL WHERE

A **WHERE clause** in SQL is a data manipulation language statement.

WHERE clauses are not mandatory clauses of SQL DML statements. But it can be used to limit the number of rows affected by a SQL DML statement or returned by a query.

Actually, it filters the records. It returns only those queries which fulfill the specific conditions.

WHERE clause is used in SELECT, UPDATE, DELETE statement etc.

1. **SELECT** column1, **column** 2, ... **column** n
2. **FROM** table_name
3. **WHERE** [conditions]
4. WHERE clause uses some conditional selection

=	equal
>	greater than
<	less than
>=	greater than or equal
<=	less than or equal
< >	not equal to

- SELECT columns FROM tables WHERE condition 1 AND condition 2;

1

```
sql> SELECT *FROM emp WHERE Department = "IT" AND Location =  
"Chennai";
```

ID	First_Name	Last_Name	Department
3	Chaitali	Tarle	IT
4	Pranjal	Patil	IT

-
- sql> **SELECT** ***FROM** emp **WHERE** Department = "IT" AND Location = "Mumbai";

ID	First_Name	Last_Name	Department
2	Anurag	Rajput	IT

SQL OR

The SQL **OR** condition is used in SQL query to create a SQL statement where records are returned when any one condition met. It can be used in a **SELECT** statement, **INSERT** statement, **UPDATE** statement or **DELETE** statement.

Let's see the syntax for the OR condition:

- SELECT** columns **FROM** tables **WHERE** condition 1 **OR** condition 2;
 sql> **SELECT** ***FROM** emp **WHERE** Department = "IT" OR Location = "Chennai";

ID	First_Name	Last_Name	Department
2	Anurag	Rajput	IT
3	Chaitali	Tarle	IT
4	Pranjal	Patil	IT

SQL SELECT DISTINCT

The **SQL DISTINCT command** is used with SELECT key word to retrieve only distinct or unique data.

In a table, there may be a chance to exist a duplicate value and sometimes we want to retrieve only unique values. In such scenarios, SQL SELECT DISTINCT statement is used.

Let's see the syntax of select distinct statement.

1. **SELECT DISTINCT** column_name ,column_name
2. **FROM** table_name;

Let's try to understand it by the table given b

Student_Name	Gender	Mobile_Number	
Rahul Ojha	Male	7503896532	
Disha Rai	Female	9270568893	
Sonoo Jaiswal	Male	9990449935	

Here is a table of students from where we want to retrieve distinct information For example: distinct home-town.

1. **SELECT DISTINCT** home_town
2. **FROM** students

Now, it will return two rows.

HOME_TOWN
Lucknow
Varanasi

SQL ALTER TABLE

The ALTER TABLE statement in Structured Query Language allows you to add, modify, and delete columns of an existing table. This statement also allows database users to add and remove various SQL constraints on the existing tables.

Any user can also change the name of the table using this statement.

ALTER TABLE ADD Column statement in SQL

In many situations, you may require to add the columns in the existing table. Instead of creating a whole table or database again you can easily add single and multiple columns using the ADD keyword.

Syntax of ALTER TABLE ADD Column statement in SQL

1. ALTER TABLE table_name ADD column_name column-definition;
2. ALTER TABLE Cars ADD Car_Model Varchar(20);

ALTER TABLE Structure

The SQL ALTER TABLE command is used to change the structure of an existing table. It helps to add or delete columns, create or destroy indexes, change the type of existing columns, or rename columns or the table itself.

It can also be used to change the comment for the table and type of the table.

Syntax:

```
ALTER TABLE <table_name>(  
ADD column1      data_type[(size)],  
ADD column2      data_type[(size)],  
...);
```

Parameters:

Name	Description
table_name	Name of the table where data is stored.

column1,column2	Name of the columns of a table.
data type	Char, varchar, integer, decimal, date and more.
size	Maximum length of the column of a table

Constraints in SQL

Constraints in SQL means we are applying certain conditions or restrictions on the database. This further means that before inserting data into the database, we are checking for some conditions. If the condition we have applied to the database holds true for the data which is to be inserted, then only the data will be inserted into the database tables.

Constraints in SQL can be categorized into two types:

- Column Level Constraint:**
 Column Level Constraint is used to apply a constraint on a single column.
- Table Level Constraint:**
 Table Level Constraint is used to apply a constraint on multiple columns.

Some of the real-life examples of constraints are as follows:

- Every person has a unique email id. This is because while creating an email account for any user, the email providing services such as Gmail, Yahoo or any other email providing service will always check for the availability of the email id that the user wants for himself. If some other user already takes the email id that the user wants, then that id cannot be assigned to another user. This simply means that no two users can have the same email ids on the same email providing service. So, here the email id is the constraint on the database of email providing services.
- Whenever we set a password for any system, there are certain constraints that are to be followed. These constraints may include the following:
 - There must be one uppercase character in the password.
 - Password must be of at least eight characters in length.

- Password must contain at least one special symbol.

Constraints available in SQL are:

1. NOT NULL
2. UNIQUE
3. PRIMARY KEY
4. FOREIGN KEY
5. CHECK
6. DEFAULT
7. CREATE INDEX

Now let us try to understand the different constraints available in SQL in more detail with the help of examples. We will use MySQL database for writing all the queries.

1. NOT NULL

- NULL means empty, i.e., the value is not available.
- Whenever a table's column is declared as NOT NULL, then the value for that column cannot be empty for any of the table's records.
- There must exist a value in the column to which the NOT NULL constraint is applied.

Syntax to apply the NOT NULL constraint during table creation:

1. **CREATE TABLE** TableName (ColumnName1 datatype NOT NULL, ColumnName2 datatype, ..., ColumnNameN datatype);
2. **Example:CREATE TABLE** student(StudentID **INT** NOT NULL, Student_FirstName **VARCHAR**(20), Student_LastName **VARCHAR**(20), Student_PhoneNumber **VARCHAR**(20), Student_Email_ID **VARCHAR**(40));

To verify that the not null constraint is applied to the table's column and the student table is created successfully, we will execute the following query:

```
sql> DESC student;
```

3.

```
mysql> DESC student;
```

Field	Type	Null	Key	Default	Extra
StudentID	int(11)	NO		NULL	
Student_FirstName	varchar(20)	YES		NULL	
Student_LastName	varchar(20)	YES		NULL	
Student_PhoneNumber	varchar(20)	YES		NULL	
Student_Email_ID	varchar(40)	YES		NULL	

5 rows in set (0.14 sec)

2. UNIQUE

- Duplicate values are not allowed in the columns to which the UNIQUE constraint is applied.
- The column with the unique constraint will always contain a unique value.
- This constraint can be applied to one or more than one column of a table, which means more than one unique constraint can exist on a single table.
- Using the UNIQUE constraint, you can also modify the already created tables.

Syntax to apply the UNIQUE constraint on a single column:

1. **CREATE TABLE** TableName (ColumnName1 datatype **UNIQUE**, ColumnName2 datatype, ..., ColumnNameN datatype);

Create a student table and apply a UNIQUE constraint on one of the table's column while creating a table.

```
sql> CREATE TABLE student(StudentID INT UNIQUE, Student_FirstName VARCHAR(20), Student_LastName VARCHAR(20), Student_PhoneNumber VARCHAR(20), Student_Email_ID VARCHAR(40));
```

```
mysql> DESC student;
```

Field	Type	Null	Key	Default	Extra
StudentID	int(11)	YES	UNI	NULL	
Student_FirstName	varchar(20)	YES		NULL	
Student_LastName	varchar(20)	YES		NULL	
Student_PhoneNumber	varchar(20)	YES		NULL	
Student_Email_ID	varchar(40)	YES		NULL	

```
5 rows in set (0.01 sec)
```

3. PRIMARY KEY

- PRIMARY KEY Constraint is a combination of NOT NULL and Unique constraints.
- NOT NULL constraint and a UNIQUE constraint together forms a PRIMARY constraint.
- The column to which we have applied the primary constraint will always contain a unique value and will not allow null values.

1. **CREATE TABLE** TableName (ColumnName1 datatype **PRIMARY KEY**, ColumnName2 datatype, ..., ColumnNameN datatype);

Example: Create a student table and apply the PRIMARY KEY constraint while creating a table.

1. mysql> **CREATE TABLE** student(StudentID **INT PRIMARY KEY**, Student_FirstName **VARCHAR**(20), Student_LastName **VARCHAR**(20), Student_PhoneNumber **VARCHAR**(20), Student_Email_ID **VARCHAR**(40));

```
mysql> DESC student;
```

Field	Type	Null	Key	Default	Extra
StudentID	int(11)	NO	PRI	NULL	
Student_FirstName	varchar(20)	YES		NULL	
Student_LastName	varchar(20)	YES		NULL	
Student_PhoneNumber	varchar(20)	YES		NULL	
Student_Email_ID	varchar(40)	YES		NULL	

```
5 rows in set (0.10 sec)
```

4. FOREIGN KEY

- A foreign key is used for referential integrity.
- When we have two tables, and one table takes reference from another table, i.e., the same column is present in both the tables and that column acts as a primary key in one table. That particular column will act as a foreign key in another table.

CREATE TABLE tablename(ColumnN1 Datatype(**SIZE**) **PRIMARY KEY**, ColumnN2 Datatype(**SIZE**), **FOREIGN KEY**(ColumnName) **REFERENCES** PARENT_TABLE_NAME(Primary_Key_ColumnName));

```
sql> CREATE TABLE employee (Emp_ID INT NOT NULL PRIMARY KEY, Emp_Name VA
```

```
RCHAR (40), Emp_Salary VARCHAR (40));
```

```
mysql> DESC employee;
```

Field	Type	Null	Key	Default	Extra
Emp_ID	int(11)	NO	PRI	NULL	
Emp_Name	varchar(40)	YES		NULL	
Emp_Salary	varchar(40)	YES		NULL	

```
3 rows in set (0.01 sec)
```

5. CHECK

- Whenever a check constraint is applied to the table's column, and the user wants to insert the value in it, then the value will first be checked for certain conditions before inserting the value into that column.
- **For example:** if we have an age column in a table, then the user will insert any value of his choice. The user will also enter even a negative value or any other invalid value. But, if the user has applied check constraint on the age column with the condition age greater than 18. Then in such cases, even if a user tries to insert an invalid value such as zero or any other value less than 18, then the age column will not accept that value and will not allow the user to insert it due to the application of check constraint on the age column.

CREATE TABLE TableName (ColumnName1 datatype **CHECK** (ColumnName1 Condition), ColumnName2 datatype, ..., ColumnNameN datatype);

1. `mysql> CREATE TABLE student(StudentID INT, Student_FirstName VARCHAR(20), Student_LastName VARCHAR(20), Student_PhoneNumber VARCHAR(20), Student_Email_ID VARCHAR(40), Age INT CHECK( Age <= 15));`

```
mysql> DESC student;
```

Field	Type	Null	Key	Default	Extra
StudentID	int(11)	YES		NULL	
Student_FirstName	varchar(20)	YES		NULL	
Student_LastName	varchar(20)	YES		NULL	
Student_PhoneNumber	varchar(20)	YES		NULL	
Student_Email_ID	varchar(40)	YES		NULL	
Age	int(11)	YES		NULL	

```
6 rows in set (0.01 sec)
```

6. DEFAULT

Whenever a default constraint is applied to the table's column, and the user has not specified the value to be inserted in it, then the default value which was specified while applying the default constraint will be inserted into that particular column.

Syntax to apply default constraint during table creation:

1. **CREATE TABLE** TableName (ColumnName1 datatype **DEFAULT** Value, ColumnName2 datatype, ..., ColumnNameN datatype);

```
mysql> CREATE TABLE student(StudentID INT, Student_FirstName VARCHAR(20), Student_LastName VARCHAR(20), Student_PhoneNumber VARCHAR(20), Student_Email_ID VARCHAR(40) DEFAULT "anuja.k8@gmail.com");
```

```
mysql> DESC student;
```

Field	Type	Null	Key	Default	Extra
StudentID	int(11)	YES		NULL	
Student_FirstName	varchar(20)	YES		NULL	
Student_LastName	varchar(20)	YES		NULL	
Student_PhoneNumber	varchar(20)	YES		NULL	
Student_Email_ID	varchar(40)	YES		anuja.k8@gmail.com	

5 rows in set (0.01 sec)

7. CREATE INDEX

CREATE INDEX constraint is used to create an index on the table. Indexes are not visible to the user, but they help the user to speed up the searching speed or retrieval of data from the database.

1. **CREATE INDEX** IndexName **ON** TableName (ColumnName 1);
 2. mysql> **CREATE INDEX** idx_StudentID **ON** student (StudentID);

```
mysql> DESC student;
```

Field	Type	Null	Key	Default	Extra
StudentID	int(11)	YES	MUL	NULL	
Student_FirstName	varchar(20)	YES		NULL	
Student_LastName	varchar(20)	YES		NULL	
Student_PhoneNumber	varchar(20)	YES		NULL	
Student_Email_ID	varchar(40)	YES		NULL	

5 rows in set (0.07 sec)

OPERATORS AND SQL Functions & views

Arithmetic operators

In Structured Query Language, the arithmetic operators are used to perform mathematical operations on the numerical values stored in the database tables. We can use these operators with the SELECT statement in SQL. We can also use the WHERE clause in the SELECT statement for performing operations on particular rows. These types of operators are used between two numerical operands for performing addition, subtraction, multiplication, and division operations.

The arithmetic operators in SQL are categorized into the following five types:

1. SQL Addition Operator (+)
2. SQL Subtraction Operator (-)
3. SQL Multiplication Operator (*)
4. SQL Division Operator (/)
5. SQL Modulus Operator (%)

1. SQL Addition Operator (+)

The SQL Addition Operator performs the addition on the numerical columns in the table. If you want to add the values of two numerical columns in the table, then you have to specify both columns as the first and second operand. You can also add the new integer value in the value of the integer column.

Syntax of SQL Addition Operator:

SELECT Column_Name_1 Addition_Operator Column_Name2 **FROM** Table
_Name;

1. **SELECT * FROM** Employee;

Employee_Id	Emp_Name	Emp_City	Emp_Salary
101	Anuj	Ghaziabad	25000

102	Tushar	Lucknow	29000
103	Vivek	Kolkata	35000
104	Shivam	Goa	22000

SELECT Emp_Salary + Emp_Bonus **AS** Emp_Total_Salary **FROM** Employee;

Emp_Total_Salary
27000
30000
37500
25000

2 SQL Subtraction Operator (-)

The SQL Subtraction Operator performs the subtraction on the numerical columns in the table. If we want to subtract the values of one numerical column from the values of another numerical column, then we have to specify both columns as the first and second operand. We can also subtract the integer value from the values of the integer column.

1. **SELECT** Column_Name_1 Subtraction_Operator Column_Name2 **FROM** Table_Name;
SELECT * FROM Employee;

Employee_Id	Emp_Name	Emp_City	Emp_Salary	Emp_Panelty
101	Anuj	Ghaziabad	25000	500
102	Tushar	Lucknow	29000	1000
103	Vivek	Kolkata	35000	700
104	Shivam	Goa	22000	500

The following query subtracts the values of the Emp_Panelty column from the Emp_Salary column of the **Employee** table using the subtraction operator:

1. **SELECT** Emp_Salary - Emp_Panelty **AS** Emp_Total_Salary **FROM** Employee;

Emp_Total_Salary
24500
28000
34300
12000

3.SQL Multiplication Operator (*)

The SQL Multiplication Operator performs the multiplication on the numerical columns in the table.If you want to multiply the values of two numerical columns, then you have to specify both columns as the first and second operand. You can also multiply the integer value with the values of an integer column.

Syntax of SQL Multiplication Operator:

1. **SELECT** Column_Name_1 Multiplication_Operator Column_Name2 **FROM** Table_Name;
2. **SELECT** Column_Name_1 Multiplication_Operator Column_Name2 **FROM** Table_Name **WHERE** Condition;
3. **SELECT** * **FROM** Cars;

Car_Number	Car_Name	Car_Amount	Car_Price
2578	Creta	3	1500000
9258	Audi	2	3000000
8233	Venue	6	900000
6214	Nexon	7	1000000

1. **SELECT** Car_Amount * Car_Price **AS** Car_Total_Price **FROM** Cars;
2. **SELECT** Car_Amount * Car_Price **AS** Car_Total_Price **FROM** Cars;

Emp_Total_Salary
4500000
6000000
5400000
7000000

4. SQL Division Operator (/)

The SQL Division operator divides the numerical values of one column by the numerical values of another column.

Syntax of SQL Division Operator:

1. **SELECT** Column_Name_1 Division_Operator Column_Name2 **FROM** Table_Name;
1. **SELECT * FROM** Cars;

Car_Number	Car_Name	Car_Amount	Car_Price
2578	Creta	3	1500000
9258	Audi	2	3000000
8233	Venue	6	900000
6214	Nexon	10	1000000

The following query divides the values of the Car_Price column by the Car_Amount column of the **Cars** table using the Multiplication operator:

1. **SELECT** Car_Price / Car_Amount **AS** One_Car_Price **FROM** Cars;

One_Car_Price
500000
1500000
150000
100000

5. SQL Modulus Operator (%)

The SQL Modulus Operator provides the remainder when the numerical values of one column are divided by the numerical values of another column.

Syntax of Modulus Operator in SQL:

1. **SELECT** Column_Name_1 Modulus_Operator Column_Name2 **FROM** Table_Name;
2. **SELECT** * **FROM** Student;

Student_Id	Student_Name	Student_Maths	Student_English
201	Anuj	30	60
202	Tushar	25	100
203	Vivek	30	90
204	Shivam	40	80

1. **SELECT** Student_English % Student_Maths **AS** Remainder **FROM** Student;

Remainder
2
4
3
2

SQL Logical Operators

The **Logical Operators** in SQL perform the Boolean operations, which give two results **True** and **False**. These operators provide **True** value if both operands match the logical condition.

Following are the various logical operators which are performed on the data stored in the SQL database tables:

1. SQL ALL operator

2. SQL AND operator
3. SQL OR operator
4. SQL BETWEEN operator
5. SQL IN operator
6. SQL NOT operator
7. SQL ANY operator
8. SQL LIKE operator

SQL ALL Operator

The ALL operator in SQL compares the specified value to all the values of a column from the sub-query in the SQL database.

This operator is always used with the following statement:

1. SELECT,
2. HAVING, and
3. WHERE.

Syntax

```
SELECT column_Name1, ..., column_NameN FROM table_Name WHERE  
column Comparison_operator ALL (SELECT column FROM tablename2)
```

Example

```
SELECT Emp_Id, Emp_Name FROM Employee_details WHERE Emp_Salary > ALL (  
SELECT Emp_Salary FROM Employee_details WHERE Emp_City = Jaipur)
```

SQL AND Operator

The **AND operator** in SQL would show the record from the database table if all the conditions separated by the AND operator evaluated to True. It is also known as the conjunctive operator and is used with the WHERE clause.

Syntax of AND operator:

1. SELECT column1, ..., columnN FROM table_Name WHERE condition1 AND condition2 AND condition3 AND AND conditionN;

This example consists of an **Employee_details** table, which has three columns **Emp_Id**, **Emp_Name**, **Emp_Salary**, and **Emp_City**.

Emp Id	Emp Name	Emp Salary	Emp City
201	Abhay	25000	Delhi
202	Ankit	45000	Chandigarh
203	Bheem	30000	Delhi
204	Ram	25000	Delhi
205	Sumit	40000	Kolkata

- Suppose, we want to access all the records of those employees from the **Employee_details** table whose salary is 25000 and the city is Delhi. For this, we have to write the following query in SQL:
1. SELECT * FROM Employee_details WHERE **Emp_Salary = 25000** OR **Emp_City = 'Delhi'**;

Here, **SQL AND operator** with WHERE clause shows the record of employees whose salary is 25000 and the city is Delhi.

SQL OR Operator

The OR operator in SQL shows the record from the table if any of the conditions separated by the OR operator evaluates to True. It is also known as the conjunctive operator and is used with the WHERE clause.

Syntax of OR operator:

1. SELECT column1, ..., columnN FROM table_Name WHERE condition1 OR condition2 OR condition3 OR OR conditionN;

This example consists of an **Employee_details** table, which has three columns **Emp_Id**, **Emp_Name**, **Emp_Salary**, and **Emp_City**.

Emp Id	Emp Name	Emp Salary	Emp City
201	Abhay	25000	Delhi
202	Ankit	45000	Chandigarh
203	Bheem	30000	Delhi
204	Ram	25000	Delhi
205	Sumit	40000	Kolkata

- If we want to access all the records of those employees from the **Employee_details** table whose salary is 25000 or the city is Delhi. For this, we have to write the following query in SQL:

1. SELECT * FROM Employee_details WHERE **Emp_Salary** = 25000 OR **Emp_City** = 'Delhi';

Here, **SQL OR operator** with WHERE clause shows the record of employees whose salary is 25000 or the city is Delhi.

SQL BETWEEN Operator

The **BETWEEN operator** in SQL shows the record within the range mentioned in the SQL query. This operator operates on the numbers, characters, and date/time operands.

1. SELECT column_Name1, column_Name2 ..., column_NameN FROM table_Name WHERE column_nameBETWEEN value1 and value2;

This example consists of an **Employee_details** table, which has three columns **Emp_Id**, **Emp_Name**, **Emp_Salary**, and **Emp_City**.

Emp Id	Emp Name	Emp Salary	Emp City
201	Abhay	25000	Delhi
202	Ankit	45000	Chandigarh
203	Bheem	30000	Delhi
204	Ram	25000	Delhi
205	Sumit	40000	Kolkata

- Suppose, we want to access all the information of those employees from the **Employee_details** table who is having salaries between 20000 and 40000. For this, we have to write the following query in SQL:

1. SELECT * FROM Employee_details WHERE Emp_Salary BETWEEN 30000 AND 45000;

SQL IN Operator

The **IN operator** in SQL allows database users to specify two or more values in a WHERE clause. This logical operator minimizes the requirement of multiple OR conditions.

This operator makes the query easier to learn and understand. This operator returns those rows whose values match with any value of the given list

1. SELECT column_Name1, column_Name2 ..., column_NameN FROM table_Name WHERE column_name IN (list_of_values);

Let's understand the below example which explains how to execute IN logical operator in SQL query:

This example consists of an **Employee_details** table, which has three columns **Emp_Id**, **Emp_Name**, **Emp_Salary**, and **Emp_City**.

Emp Id	Emp Name	Emp Salary	Emp City
201	Abhay	25000	Delhi
202	Ankit	45000	Chandigarh
203	Bheem	30000	Delhi
204	Ram	25000	Delhi
205	Sumit	40000	Kolkata

- Suppose, we want to show all the information of those employees from the **Employee_details** table whose **Employee Id** is 202, 204, and 205. For this, we have to write the following query in SQL:

1. SELECT * FROM Employee_details WHERE Emp_Id IN (202, 204, 205);

Here, we used the **SQL IN operator** with the Emp_Id column.

- Suppose, we want to show all the information of those employees from the **Employee_details** table whose **Employee Id** is not equal to 202 and 205. For this, we have to write the following query in SQL:

1. SELECT * FROM Employee_details WHERE Emp_Id NOT IN (202,205);

2.

Here, we used the **SQL NOT IN operator** with the Emp_Id column.

SQL NOT Operator

The **NOT operator** in SQL shows the record from the table if the condition evaluates to false. It is always used with the WHERE clause.

Syntax of NOT operator:

1. SELECT column1, column2 ..., columnN FROM table_Name WHERE NOT condition;
2. **Let's understand the below example which explains how to execute NOT logical operator in SQL query:**

3. This example consists of an **Employee_details** table, which has four columns **Emp_Id**, **Emp_Name**, **Emp_Salary**, and **Emp_City**.

Emp Id	Emp Name	Emp Salary	Emp City
201	Abhay	25000	Delhi
202	Ankit	45000	Chandigarh
203	Bheem	30000	Delhi
204	Ram	25000	Delhi
205	Sumit	40000	Kolkata

1. `SELECT * FROM Employee_details WHERE NOT Emp_City = 'Delhi' ;`

In this example, we used the **SQL NOT operator** with the Emp_City column.

SQL LIKE Operator

The **LIKE operator** in SQL shows those records from the table which match with the given pattern specified in the sub-query.

The percentage (%) sign is a wildcard which is used in conjunction with this logical operator.

This operator is used in the WHERE clause with the following three statements:

1. SELECT statement
2. UPDATE statement
3. DELETE statement

Syntax of LIKE operator:

```
SELECT column_Name1, column_Name2 ..., column_NameN FROM table_Name WHERE c  
olumn_name LIKE pattern;
```

Let's understand the below example which explains how to execute LIKE logical operator in SQL query:

This example consists of an **Employee_details** table, which has four columns **Emp_Id**, **Emp_Name**, **Emp_Salary**, and **Emp_City**.

Emp Id	Emp Name	Emp Salary	Emp City
201	Sanjay	25000	Delhi
202	Ajay	45000	Chandigarh
203	Saket	30000	Delhi
204	Abhay	25000	Delhi
205	Sumit	40000	Kolkata

- If we want to show all the information of those employees from the **Employee_details** whose name starts with 's'. For this, we have to write the following query in SQL:

1. SELECT * FROM Employee_details WHERE Emp_Name LIKE 's%';

In this example, we used the SQL LIKE operator with **Emp_Name** column because we want to access the record of those employees whose name starts with s.

- If we want to show all the information of those employees from the **Employee_details** whose name ends with 'y'. For this, we have to write the following query in SQL:

1. SELECT * FROM Employee_details WHERE Emp_Name LIKE '%y';

SQL Functions

Single Row Function

Single row function in SQL are the ones who work on a single row and return one output per row. For example, length and case conversion functions are single row functions.

Types of **Single Row Function**

1 Character Function

2. Number Function
3. Date Function
4. Conversion Function
5. General Function
6. Time Stamp Function

1 Character Function

A) Case Manipulation Function

a) Upper

Example:- Sql> select upper('sybcs')from dual;

o/p:- UPPER

SYBCS

b) Lower

Example:- Sql> select Lower('SYBCS')from dual;

o/p:- UPPER

sybcs

c) init cap

Example:- Sql> select init cap ('sybcs') as name from dual;

o/p:- Sybcs

B) String Manipulation Function

a)length

Example:- SQL> Select length('MGM')From dual;

O/P:-

Length (MGM')

03

b)concat

Example:- SQL>select concat('India','Country')from dual;

O/P:- India country

c)lpad Example:- 1) SQL>select lpad('India',9)from dual;

o/p:- India

2) SQL>select lpad('India',9,'***')from dual; o/p:-*** India

d)rpadd Example:- 1) SQL>select rpad('India',9)from dual;

o/p: India

2) SQL>select rpad('India',9,'***')from dual; o/p:- India***

e)ltrim Example:- 1) SQL>select ltrim('Nature','N')from dual;

O/P:- ature

2)) SQL>select ltrim('NNNNNatureNNN','N');

O/P:- atureNNN

1) SQL>select ltrim('Nature','Na'from dual;;

O/P:- ture

f)rtrim Example:- 1) SQL>select rtrim('Nature','re')from dual;

O/P:- Natu

g)trim Example:- 1) SQL>select trim('N','NNNatureNN')from dual;

O/P:- ature

h)translate

Example:- 1) SQL>select translate('Manisha','ish',123)from dual;

O/P:- Man123a

i)Replace 1) SQL>select Replace('Manisha','Mani',1234)from dual;

O/P:- 1234sha

j)Substr SQL>select substr('Ankit Kumar',3,7) from dual;

O/P:-kit kum

k)instr SQL>select instr('Ankit Kumar','k') from dual;
O/P:-3

l)reverse SQL>select reverse('Ankit') from dual;
O/P:-tiknA

2 Number Function

Abs,cell,floor,power,mod,sqrt,sign,len,ln,exp,trunk,round,sin,

- a) ABS :- Example SQL> select abs(34.45)from dual;
O/P:- 34.45
- b) Cell :- Example:- SQL> select cell(123.34) from dual;
O/P :- 123
- c) Floor :- Example:- SQL>select floor(123.34)from dual; O/P :- 123
- d) Sqrt :- Exam:- SQL> select sqrt(49)from dual; o/p 7
- e) Power:- Exam:- SQL> select power (6,4)from dual; O/p 1296
- f) Mod :- Examp:- SQL> select mod(5,2) from dual; O/P 1
- g)

3 Date Function

Example SQL> select sysdate from dual; O/P 13/Jan/2023

Example SQL> select current_date from dual; O/P 13/Jan/2023

Example SQL> select add_months(sysdate,3) from dual; O/P
13/April/2023

Example SQL> select last_day(sysdate) from dual; O/P 31/jan/2023

Example SQL> select last_day('1/jan/2023') from dual; O/P 31/jan/2023

Example SQL> select next_day(current_date,'Monday') from dual; O/P
16/jan/2023

4) Conversion Function

SQL> create table sample(Phone varchar(20));

Sql> insert into sample values(55);

55

Sql>select to_number(phone)from sample;

55

```
SQL> select phone*2 from sample;
```

110

```
SQL> select phone+2 from sample;
```

57

```
SQL> select phone-2 from sample;
```

53

```
SQL> select phone/2 from sample;
```

18.33

```
SQL> select '04-April-2019' from sample;
```

04-April-2019

04-April-2019

04-April-2019'

```
SQL> select to_char(sysdate,'dd-mon-yyyy') from dual;
```

13-jan-2023

```
SQL> select to_char(sysdate,'dd-mon-yyyy day month w') from dual;
```

13-jan-2023 Friday January 2

5 General Function

- 1 NVL()
2. NVL2()
3. DECODE()
4. COALESCE()
5. LNNVL()

1 NVL functions:-

This is one of the functions of SQL extensively used in Structured Query Language (SQL).

This function can hold two input values only. If input values are more than 2 then an error is returned. This functions returns the first NOT NULL value when searched in the function.

If both the inputs are NULL, then there is no output for this function.

Converts null values into actual value.

```
SQL> Create table EMP (Name varchar(20),Esalary int,Code int);
```

```
SQL> insert into EMP values('mavi',6000,002);
```

```
SQL> insert into EMP values('rr',5000);
```

```
SQL> insert into EMP values('tt',5000);
```

```
SQL> Select Name,Esalary,code from Employee;
```

Name	Esalary	Code
Mavi	6000	002
Rr	5000	
Tt	5000	

```
SQL> Select Name,Esalary,NVL(code,0) from Employee;
```

Name	Esalary	Code
Mavi	6000	002
Rr	5000	00
Tt	5000	00

```
SQL> select NVL(1, 2) from dual;
```

NVL(1,2)

1

```
SQL> select NVL(NULL, 1) from dual;
```

NVL(NULL,1)

1

```
SQL> select NVL(1.029384, 1.029384) from dual;
```

NVL(1.029384,1.029384)

1.029384

```
SQL> select NVL(NULL, 1.029384) from dual;
```

NVL(NULL,1.029384)

1.029384

```
SQL> select NVL('JOE', 'ROOT') from dual;    o/p JOE
```

2.) NVL2()

This is one of the functions of SQL extensively used in Structured Query Language (SQL). This function can hold three input values only. If input values are more than three then an error is returned. This function returns the first value after NOT NULL value is found, when searched in the function. If the first value is NOT NULL, second value is returned. If the first value is NULL and the second value is NOT NULL, third value is returned. The returned value can also be a NULL value too.

Syntax

NVL2(input 1, input 2, input 3)

1. SQL> **select** NVL2(1, 2, 3) **from** dual; o/p 2
1. SQL> **select** NVL2(NULL, 'Pitersen', ' SA / ENG ') **from** dual; o/p SA / ENG

3.) DECODE()

This is also one of the expressions used in SQL. This Decode expression is used as IF, ELSE IF, ELSE IF Ladder style. This decode works on the basis of the condition specified. Any kind of operation specified is going to work here. The input types must be chosen based on the data types specified.

Syntax

DECODE (**column name**, number 1 **to** be searched, result 1 **to** be updated
d
 , number 2 **to** be searched, result 2 **to** be updated
 , number 3 **to** be searched, result 3 **to** be updated

 number n **to** be searched, result n **to** be updated , **default**)

SQL > **select** * **from** ipla;

SID	SNAME	SAL	AGE
1	mahi	12	40
2	kohli	14	33
3	DK	6.25	33
4	warner	6.75	33
5	rahul	16	29
6	pandya	14	27

SQL > **SELECT** Sname, sid, sal,
DECODE (sid, 1, 1.5*sal,
2, 4*sal,
3, 9*sal,
4, 10.25*sal,
sal)
"REVISED SALARY"
from ipla ;

1.	SNAME	SID	SAL	REVISED SALARY
2.	---	-----	-----	-----
3.	mahi	1	12	18
4.	kohli	2	14	56
5.	DK	3	6.25	56.25
6.	warner	4	6.75	69.1875
7.	rahul	5	16	16
8.	pandya	6	14	14

4.) COALESCE()

This also one of the expression used in SQL. This expression works similar to NVL () expression. The only difference it can accept inputs greater than two. It returns the first NOT NULL input element.

The input data type can be anything. The inputs can be int, float, string, character, number, etc.

Syntax

1. COALESCE (input 1, input 2, input 3,, input n)
2. SQL> select COALESCE(NULL, 1) from dual;
- 3.
4. COALESCE(NULL,1)
5. -----
6. 1
7. SQL> select COALESCE(1, 2, 2) from dual;
- 8.
9. COALESCE(1,2,2)
10. -----
11. 1

5.) LNNVL()

This is one of the function of SQL which is used in SQL. This is used to convert True to False and False to True. The LNNVL () function has the capacity to hold a condition. This makes the condition go reverse. If the condition is SID = 2. Then LNNVL (SID = 2) is equivalent to SID != 2.

Syntax LNNVL (Condition)

1. SQL> select * from ipla;
- 2.
3. SID SNAME SAL AGE
4. ---
5. 1 mahi 12 40
6. 2 kohli 14 33

7.	3	DK	6.25	33
8.	4	warner	6.75	33
9.	5	rahul	16	29
10.	6	pandya	14	27
11.	7	Tim David	8.25	26

12.

13. 7 rows selected.

14.

15. SQL> select * from ipla where sid=2;

16.	SID	SNAME	SAL	AGE
17.	---	-----	----	----
18.	2	kohli	14	33

19.

20. SQL> select * from ipla where LNNVL (sid = 2) ;

21.

22.	SID	SNAME	SAL	AGE
23.	-----	-----	----	----
24.	1	mahi	12	40
25.	3	DK	6.25	33
26.	4	warner	6.75	33
27.	5	rahul	16	29
28.	6	pandya	14	27
29.	7	Tim David	8.25	26

30. 6 rows selected.

Multiple Row Functions

Aggrgate Function/ Group /Multi row Function

N number of records then it will produce one output.

Count(),Sum(),max(),

1) Count()

SQL>select count(EID) from Emp; o/p 3

SQL>select count(*) from Emp; o/p 3 (All records)

2) Max()

SQL>select max(EID) from Emp; o/p 300 (maximum number from all records)

3)Min()

SQL>select min(EID) from Emp; o/p 100 (manimum number from all records)

4)Sum()

SQL>select Sum(EID) from Emp; o/p 800 (addition of all records from field)

5)AVG()

SQL>select avg(EID) from Emp; o/p 12.5 (average number from all records)

6)stddev()

SQL>select stddev(EID) from Emp; o/p 56.23 (Standard deviation of number)

7)Varience()

SQL>select varience(EID) from Emp; o/p 5645.23 (variance of number)

SQL View

SQL provides the concept of VIEW, which hides the complexity of the data and restricts unnecessary access to the database. It permits the users to access only a particular column rather than the whole data of the table.

The **View** in the Structured Query Language is considered as the virtual table, which depends on the result-set of the predefined SQL statement.

Like the SQL tables, Views also store data in rows and columns, but the rows do not have any physical existence in the database.

Any database administrator and user can easily create the View by selecting the columns from one or more database tables. They can also delete and update the views according to their needs

Create a SQL View

You can easily create a View in Structured Query Language by using the CREATE VIEW statement. You can create the View from a single table or multiple tables.

Syntax to Create View from Single Table

```
CREATE VIEW View_Name AS
SELECT Column_Name1, Column_Name2, ....., Column_NameN
FROM Table_Name
WHERE condition;
```

In the syntax, View_Name is the name of View you want to create in SQL. The SELECT command specifies the rows and columns of the table, and the WHERE clause is optional, which is used to select the particular record from the table.

Syntax to Create View from Multiple Tables

You can create a View from multiple tables by including the tables in the SELECT statement.

```
CREATE VIEW View_Name AS
SELECT Table_Name1.Column_Name1, Table_Name1.Column_Name2, Table_Name1.
Column_Name2, ....., Table_NameN.Column_NameN
FROM Table_Name1, Table_Name2, ....., Table_NameN
WHERE condition;
```

Example to Create a View from Single table

Let's consider the **Student_Details** table, which consists of Stu_ID, Stu_Name, Stu_Subject, and Stu_Marks columns. The data of the Student_Details is shown in the following table:

Student_ID	Stu_Name	Stu_Subject	Stu_Marks
1001	Akhil	Math	85
1002	Balram	Science	78
1003	Bheem	Math	87
1004	Chetan	English	95
1005	Diksha	Hindi	99
1006	Raman	Computer	90
1007	Sheetal	Science	68

Table: Student_Details

Suppose, you want to create a view with Stu_ID, Stu_Subject, and Stu_Marks of those students whose marks are greater than 85. For this issue, you have to type the following query:

```
CREATE VIEW Student_View AS
SELECT Student_ID, Stu_Subject, Stu_Marks
FROM Student_Details
WHERE Stu_Marks > 85;
```

```
Select * FROM Student_View;
```

Output:

Student_ID	Stu_Subject	Stu_Marks
1001	Math	85
1003	Math	87
1004	English	95

1005	Hindi	99
1006	Computer	90

Update an SQL View

We can also modify existing data and insert the new record into the view in the Structured Query Language. A view in SQL can only be modified if the view follows the following conditions:

1. You can update that view which depends on only one table. SQL will not allow updating the view which is created more than one table.
2. The fields of view should not contain NULL values.
3. The view does not contain any subquery and DISTINCT keyword in its definition.
4. The views cannot be updatable if the SELECT statement used to create a View contains JOIN or HAVING or GROUP BY clause.
5. If any field of view contains any SQL aggregate function, you cannot modify the view.

Syntax to Update a View

```
CREATE OR REPLACE VIEW View_Name AS
SELECT Column_Name1, Column_Name2, ....., Column_NameN
FROM Table_Name
WHERE condition;
```

Example to Update a View

If we want to update the above Student_View and add the Stu_Name attribute from the Student table in the view, you have to type the following Replace query in SQL:

```
CREATE OR REPLACE VIEW Student_View AS
SELECT Student_ID, Stu_Name, Stu_Subject, Stu_Marks
FROM Student_Details
```

WHERE Stu_Subject = 'Math';

The above statement updates the existing Student_View and updates the data based on the SELECT statement.

Now, you can see the virtual table by typing the following query:

SELECT * FROM Student_View;

Output:

Student_ID	Stu_Name	Stu_Subject	Stu_Marks
1001	Akhil	Math	85
1003	Bheem	Math	87

Sorting and Grouping Data and Joining tables & subqueries in Oracle

What is Sorting:-

The SQL **ORDER BY** clause is used to sort the data in ascending or descending order, based on one or more columns. Some databases sort the query results in an ascending order by default.

ORDER BY

The **ORDER BY** command is used to sort the result set in ascending or descending order.

The **ORDER BY** command sorts the result set in ascending order by default. To sort the records in descending order, use the **DESC** keyword.

The following SQL statement selects all the columns from the "Customers" table, sorted by the "CustomerName" column:

Syntax

The basic syntax of the ORDER BY clause which would be used to sort the result in an ascending or descending order is as follows –

```
SELECT column-list  
FROM table_name  
[WHERE condition]  
[ORDER BY column1, column2, .. columnN] [ASC | DESC];
```

Example

```
SELECT * FROM Customers  
ORDER BY CustomerName;
```

Example

Consider the CUSTOMERS table having the following records –

ID	NAME	AGE	ADDRESS	SALARY
1	Ramesh	32	Ahmedabad	2000.00
2	Khilan	25	Delhi	1500.00
3	kaushik	23	Kota	2000.00
4	Chaitali	25	Mumbai	6500.00
5	Hardik	27	Bhopal	8500.00
6	Komal	22	MP	4500.00
7	Muffy	24	Indore	10000.00

```
SQL> SELECT * FROM CUSTOMERS  
ORDER BY NAME, SALARY;
```

This would produce the following result –

4	Chaitali	25	Mumbai	6500.00
5	Hardik	27	Bhopal	8500.00
3	kaushik	23	Kota	2000.00
2	Khilan	25	Delhi	1500.00
6	Komal	22	MP	4500.00
7	Muffy	24	Indore	10000.00
1	Ramesh	32	Ahmedabad	2000.00

ORDER BY Descending order

The following code block has an example, which would sort the result in a descending order by NAME.

```
SQL> SELECT * FROM CUSTOMERS  
ORDER BY NAME DESC;
```

This would produce the following result –

ID	NAME	AGE	ADDRESS	SALARY
1	Ramesh	32	Ahmedabad	2000.00
7	Muffy	24	Indore	10000.00
6	Komal	22	MP	4500.00
2	Khilan	25	Delhi	1500.00
3	kaushik	23	Kota	2000.00
5	Hardik	27	Bhopal	8500.00
4	Chaitali	25	Mumbai	6500.00

Group By:-

The **GROUP BY clause** is used in SQL queries to organize data that have the same attribute values. Usually, we use it with the **SELECT statement**. It is always to remember that we have to place the GROUP BY clause after the **WHERE clause**. Additionally, it is placed before the ORDER BY clause.

We can often use this clause in collaboration with aggregate functions like SUM, AVG, MIN, MAX, and COUNT to produce summary reports from the database. It's important to remember that the attribute in this clause must appear in the SELECT clause, not under an aggregate function. If we do so, the query would be incorrect. As a result, the GROUP BY clause is always used in conjunction with the SELECT clause. The query for the GROUP BY clause is grouped query, and it returns a single row for each grouped object.

The following is the syntax to use GROUP BY clause in a SQL statement:

```
SELECT column_name, function(column_name)  
FROM table_name  
WHERE condition  
GROUP BY column_name;
```

Let us understand how the GROUP BY clause works with the help of an example. Here we will demonstrate it with the same table.

The **GROUP BY** statement groups rows that have the same values into summary rows, like "find the number of customers in each country".

The **GROUP BY** statement is often used with aggregate functions (**COUNT()**, **MAX()**, **MIN()**, **SUM()**, **AVG()**) to group the result-set by one or more columns.

```
1)SELECT COUNT(CustomerID), Country
   FROM Customers
   GROUP BY Country;
```

```
2)SELECT COUNT(CustomerID), Country
   FROM Customers
   GROUP BY Country
   ORDER BY COUNT(CustomerID) DESC;
```

HAVING Clause in SQL

The HAVING clause places the condition in the groups defined by the GROUP BY clause in the SELECT statement.

This SQL clause is implemented after the 'GROUP BY' clause in the 'SELECT' statement.

This clause is used in SQL because we cannot use the WHERE clause with the SQL aggregate functions. Both WHERE and HAVING clauses are used for filtering the records in SQL queries.

Syntax of HAVING clause in SQL

1. **SELECT** column_Name1, column_Name2,, column_NameN aggregate_function_name(column_Name) **FROM** table_name **GROUP BY** column_Name1 **HAVING** condition;

Examples of HAVING clause in SQL

In this article, we have taken the following four different examples which will help you how to use the HAVING clause with different SQL aggregate functions:

Example 1: Let's take the following **Employee** table, which helps you to analyze the HAVING clause with SUM aggregate function:

Emp_Id	Emp_Name	Emp_Salary	Emp_City
201	Abhay	2000	Goa
202	Ankit	4000	Delhi
203	Bheem	8000	Jaipur
204 Ram	2000	Goa	
205	Sumit	5000	Delhi

If you want to add the salary of employees for each city, you have to write the following query:

1. **SELECT** SUM(Emp_Salary), Emp_City **FROM** Employee **GROUP BY** Emp_City;

The output of the above query shows the following output:

SUM(Emp_Salary)	Emp_City
4000	Goa
9000	Delhi
8000	Jaipur

SELECT SUM(Emp_Salary), Emp_City **FROM** Employee **GROUP BY** Emp_City **HAVING** SUM(Emp_Salary)> 5000;

The output of the above SQL query shows the following table in the output:

SUM(Emp_Salary)	Emp_City
9000	Delhi

8000	Jaipur
------	--------

What is JOIN:- If you want to access more than one table through a select statement.

If you want to combine two or more table then SQL JOIN statement is used .it combines rows of that tables in one table and one can retrieve the information by a SELECT statement. The joining of two or more tables is based on common field between them.

SQL INNER JOIN also known as simple join is the most common type of join.

SQL JOIN

As the name shows, JOIN means *to combine something*. In case of SQL, JOIN means **"to combine two or more tables"**.

The SQL JOIN clause takes records from two or more tables in a database and combines it together.

ANSI standard SQL defines five types of JOIN :

1. inner join,
2. left outer join,
3. right outer join,
4. full outer join, and
5. cross join.

1)Inner join

The **INNER JOIN** keyword selects records that have matching values in both tables.

INNER JOIN Syntax

```

SELECT column_name(s)
  FROM table1
  INNER JOIN table2
    ON table1.column_name = table2.column_name;

```

Example:-

SQL> Select *from Mgm_staff;

ID	Staff_name	Staff_add
12	Rasika Sharma	Bombey
13	Mansi Varma	Pune
14	Aryan Shinde	Nanded
33	Rajesh Khanna	Delhi

SQL> Select *from Payment_mgm;

ID	Pstaff_Name	Pstaff_add
33	Rajesh Khanna	Delhi
600	Rani Mukhrjee	Bombey
490	Sahil KApur	Pune

```

SQL> select staffed,staff_name,staff_add from mgm_staff
      INNER JOIN payment_mgm on
      mgm.staff.staffID=payment_mgm.ID

```

OUT PUT:-

Staffid	Staff_name	Staff_add
---------	------------	-----------

33	Rajesh Khanna	Delhi
----	---------------	-------

SQL LEFT JOIN Keyword

The **LEFT JOIN** keyword returns all records from the left table (table1), and the matching records from the right table (table2). The result is 0 records from the right side, if there is no match.

SQL>select staffId,staff_name,staff_add from mgm_staff
LEFT JOIN ON mgm_staff.staffID=payment_mgm.ID;

OUTPUT:-

StaffID	Staff_Name	Staff_add
33	Rajesh khanna	Delhi
12	Rasika Sharma	Bombey
13	Mansi Varma	Pune
14	Aryan Shinde	Nanded

SQL RIGHT JOIN Keyword

The **RIGHT JOIN** keyword returns all records from the right table (table2), and the matching records from the left table (table1). The result is 0 records from the left side, if there is no match.

RIGHT JOIN Syntax

```
SELECT column_name(s)
FROM table1
RIGHT JOIN table2
ON table1.column_name = table2.column_name;
```

SQL>select staffId,staff_name,staff_add from mgm_staff
RIGHT JOIN ON mgm_staff.staffID=payment_mgm.ID;

OUT PUT:-

Staffid	Staff_name	Staff_add
33	Rajesh Khanna	Delhi

FULL OUTER JOIN

The **FULL OUTER JOIN** command returns all rows when there is a match in either left table or right table.

The following SQL statement selects all customers, and all orders:

```
SELECT Customers.CustomerName, Orders.OrderID
FROM Customers
FULL OUTER JOIN Orders ON Customers.CustomerID=Orders.CustomerID
ORDER BY Customers.CustomerName;
```

Note: The **FULL OUTER JOIN** keyword returns all the rows from the left table (Customers), and all the rows from the right table (Orders). If there are rows in "Customers" that do not have matches in "Orders", or if there are rows in "Orders" that do not have matches in "Customers", those rows will be listed as well.

SQL CROSS JOIN Keyword

The **CROSS JOIN** keyword returns all records from both tables (table1 and table2).

CROSS JOIN Syntax

```
SELECT column_name(s)
FROM table1
CROSS JOIN table2;
```


Example

```
SELECT Customers.CustomerName, Orders.OrderID
FROM Customers
CROSS JOIN Orders;
```

Note: The **CROSS JOIN** keyword returns all matching records from both tables whether the other table matches or not. So, if there are rows in "Customers" that do not have matches in "Orders", or if there are rows in "Orders" that do not have matches in "Customers", those rows will be listed as well.

If you add a **WHERE** clause (if table1 and table2 has a relationship), the **CROSS JOIN** will produce the same result as the **INNER JOIN** clause:

Example

```
SELECT Customers.CustomerName, Orders.OrderID
FROM Customers
CROSS JOIN Orders
WHERE Customers.CustomerID=Orders.CustomerID;
```

SQL Self Join

A self join is a regular join, but the table is joined with itself.

Self Join Syntax

```
SELECT column_name(s)
FROM table1 T1, table1 T2
WHERE condition;
```

In this tutorial we will use the well-known Northwind sample database.

Below is a selection from the "Customers" table:

CustomerID	CustomerName	ContactName	Address
------------	--------------	-------------	---------

1	Alfreds Futterkiste	Maria Anders	Obere Str. 57
2	Ana Trujillo Emparedados y helados	Ana Trujillo	Avda. de la Constituci 2222
3	Antonio Moreno Taquería	Antonio Moreno	Mataderos 23

The following SQL statement matches customers that are from the same city:

Example

```
SELECT A.CustomerName AS CustomerName1,
       B.CustomerName AS CustomerName2, A.City
FROM Customers A, Customers B
WHERE A.CustomerID <> B.CustomerID
AND A.City = B.City
ORDER BY A.City;
```